

UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE POSTGRADO Y POSTÍTULO

DETECCIÓN DE OUTLIERS EN CURVAS DE LUZ DE SUPERNOVAS CON VQ-VAE Y GENERACIÓN DE EJEMPLOS CONTRAFACTUALES

TESIS PARA OPTAR AL GRADO DE MAGÍSTER EN CIENCIAS DE LA INGENIERÍA, MENCIÓN ÁREA DE LA INGENIERÍA

MEMORIA PARA OPTAR AL TÍTULO DE INGENIERO

JAVIERA PALOMINOS CLARKE

PROFESOR GUÍA:
PROFESOR GUÍA

MIEMBROS DE LA COMISIÓN:
PROFESOR 2
PROFESOR 3

SANTIAGO DE CHILE
2026

RESUMEN DE LA MEMORIA PARA OPTAR
AL TÍTULO DE MAGÍSTER EN CIENCIAS
DE LA INGENIERÍA
POR: JAVIERA PALOMINOS CLARKE
FECHA: 2026
PROF. GUÍA: NOMBRE PROFESOR

DETECCIÓN DE OUTLIERS EN CURVAS DE LUZ DE SUPERNOVAS CON VQ-VAE Y GENERACIÓN DE EJEMPLOS CONTRAFACTUALES

En este trabajo se presenta un pipeline de detección de anomalías y generación de contrafactuales para curvas de luz de supernovas del dataset ELAsTiCC-1. El enfoque convierte las curvas de luz multibanda en imágenes 2grid y utiliza un autoencoder con cuantización vectorial (VQ-VAE) con un prior tipo MaskGIT para detectar outliers y generar explicaciones contrafactuales interpretables.

Agradecimientos

Tabla de Contenido

Capítulo 1

Introducción

1.1. Planteamiento del problema

La detección de anomalías en astronomía de dominio temporal es clave para identificar fenómenos poco frecuentes o desconocidos. La operación del Observatorio Vera C. Rubin incrementará de forma sustantiva el volumen y la complejidad de los datos fotométricos multibanda, caracterizados por muestreo irregular, relaciones entre bandas y gran variabilidad intrínseca. Este contexto plantea dos desafíos complementarios: (i) seleccionar de manera eficiente objetos potencialmente interesantes dentro de un flujo masivo de alertas y (ii) comprender, con criterios reproducibles, por qué dichos objetos se apartan del comportamiento esperado.

No basta con marcar un ejemplo como atípico; es necesario explicarlo. Entre las aproximaciones de interpretabilidad, las explicaciones contrafactuales ofrecen un marco explícito: describen modificaciones mínimas y plausibles de los datos que harían que el ejemplo dejara de ser considerado anómalo. Estas explicaciones facilitan la priorización de seguimientos y la contrastación de hipótesis, al vincular la detección con una narrativa concreta de cambio en las curvas de luz.

1.2. Justificación

El dataset ELAsTiCC-1, desarrollado en el contexto del Observatorio Vera C. Rubin, contiene cerca de 1.4 millones de curvas de luz simuladas de 17 tipos de supernovas. La escala y complejidad de este dataset hacen inviable la inspección manual, y los métodos de clasificación supervisada requieren etiquetas que en la práctica no siempre estarán disponibles en tiempo real. Un sistema de detección de anomalías no supervisado, capaz además de generar explicaciones interpretables, es fundamental para el flujo de trabajo de la astronomía de grandes volúmenes de datos.

Desde el punto de vista metodológico, la representación de curvas de luz multibanda como imágenes 2grid —donde las seis bandas fotométricas se organizan como subgráficos apilados— permite aprovechar arquitecturas convolucionales estándar y capturar de forma natural la correlación temporal entre bandas, eliminando la necesidad de manejar muestreo irregular.

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar un pipeline de detección de anomalías e interpretación contrafactual para curvas de luz de supernovas del dataset ELAsTiCC-1, basado en una representación discreta del espacio latente aprendida con un autoencoder con cuantización vectorial (VQ-VAE) y un modelo prior tipo MaskGIT.

1.3.2. Objetivos específicos

1. Diseñar e implementar una representación 2grid de curvas de luz multibanda como imágenes RGB de 256×256 píxeles, con normalización por objeto que garantice comparabilidad entre supernovas de distinta luminosidad.

2. Entrenar un VQ-VAE con codebook EMA y mecanismo de codebook reset sobre el dataset ELAsTiCC-1, evaluando la calidad de reconstrucción y la utilización del espacio latente discreto.

3. Estudiar el efecto de agregar clasificación supervisada sobre el espacio cuantizado $\mathbf{z}_q$, evaluando si la organización semántica inducida en el codebook mejora la interpretabilidad de los contrafactuales generados.

4. Entrenar un prior tipo MaskGIT sobre las secuencias de tokens del VQ-VAE y utilizarlo para asignar scores de anomalía y generar contrafactuales mediante completado condicional de tokens.

5. Evaluar el pipeline de detección de anomalías y la calidad de los contrafactuales generados según criterios de plausibilidad, parsimonia y consistencia temporal entre bandas.

Capítulo 2

Marco Teórico

2.1. Detección de anomalías en astronomía de dominio temporal

En astronomía, detectar anomalías en curvas de luz permite priorizar objetos potencialmente nuevos, raros o mal comprendidos. La próxima operación del Observatorio Vera C. Rubin incrementará de manera sustantiva el volumen y la cadencia de datos multibanda, con muestreo irregular, errores heterocedásticos y gaps. Bajo estas condiciones, los enfoques tradicionales (reglas heurísticas, umbrales simples o modelos univariados) resultan insuficientes. Se requieren métodos capaces de modelar patrones complejos y, además, explicar por qué un ejemplo es atípico para apoyar decisiones de seguimiento.

2.2. Representación de curvas de luz como imágenes

La representación de series de tiempo como imágenes ha sido explorada en distintos contextos de aprendizaje automático, con el objetivo de aprovechar la capacidad de las redes convolucionales para detectar patrones espaciales. En este trabajo, cada curva de luz multibanda se convierte en una imagen 2grid de 256×256 píxeles RGB, donde las seis bandas fotométricas (u, g, r, i, z, Y) se disponen verticalmente como subgráficos apilados. El eje horizontal codifica el tiempo en días julianos modificados (MJD), y el eje vertical el flujo fotométrico.

Para que las curvas sean visibles y comparables entre objetos de distinta luminosidad, el tiempo MJD y el flujo fotométrico de cada supernova se normalizan individualmente al rango $[0, 1]$ antes de graficar. Sin esta normalización, las curvas quedan fuera del área visible de la imagen porque los valores reales (MJD 60.000, flujo -44 a 95) no corresponden a los límites del gráfico.

Esta representación presenta ventajas concretas frente al uso directo de series de tiempo: elimina la necesidad de manejar muestreo irregular, permite codificar de forma natural la correlación entre bandas como patrones espaciales en la imagen, y habilita el uso de arquitecturas convolucionales estándar sin modificaciones. La conversión se realiza con matplotlib usando `figsize=[2.56, 2.56]` pulgadas y DPI=100, produciendo imágenes de exactamente 256×256 píxeles.

2.3. Autoencoder con cuantización vectorial (VQ-VAE)

2.3.1. Descripción general

El autoencoder con cuantización vectorial (VQ-VAE, *Vector Quantized Variational Autoencoder*) fue propuesto por van den Oord et al. (2017) como un modelo generativo que aprende representaciones discretas del espacio latente. A diferencia de los VAE estándar que producen representaciones continuas mediante el truco de la reparametrización, el VQ-VAE mapea cada vector latente a su vecino más cercano en un codebook de embeddings aprendidos. Esta discretización permite modelar distribuciones multimodales complejas sin colapso posterior y produce representaciones interpretables como secuencias de tokens, lo que facilita el uso de modelos de lenguaje sobre el espacio latente.

El VQ-VAE consta de tres componentes principales: un encoder, un codebook y un decoder.

2.3.2. Encoder

El encoder E_ϕ transforma una imagen de entrada $\mathbf{x} \in \mathbb{R}^{3\times256\times256}$ en un mapa de features continuo $\mathbf{z}_e \in \mathbb{R}^{D\times H\times W}$, donde D es la dimensión del embedding y $H \times W$ es la resolución espacial del espacio latente.

En este trabajo, el encoder utiliza tres capas convolucionales con stride 2 y tamaño de kernel 4, que reducen la resolución espacial de 256×256 a 32×32, seguidas de dos bloques residuales (ResBlocks) que refinan las representaciones sin alterar la resolución:

$$256 \times 256 \xrightarrow{\text{Conv}(3\to64,\, s=2)} 128 \times 128 \tag{2.1}$$

$$128 \times 128 \xrightarrow{\text{Conv}(64\to128,\, s=2)} 64 \times 64 \tag{2.2}$$

$$64 \times 64 \xrightarrow{\text{Conv}(128\to256,\, s=2)} 32 \times 32 \tag{2.3}$$

$$32 \times 32 \xrightarrow{\text{ResBlock}\times2} 32 \times 32 \tag{2.4}$$

El resultado es un tensor $\mathbf{z}_e \in \mathbb{R}^{256\times32\times32}$, es decir, 1024 vectores de dimensión 256, uno por cada posición espacial 32×32. Cada uno de estos vectores representa un parche de 8×8 píxeles de la imagen original.

2.3.3. Cuantización vectorial con EMA

El codebook es un conjunto de $K = 512$ vectores aprendidos $\{\mathbf{e}_k\}_{k=1}^{K} \subset \mathbb{R}^{256}$. Para cada vector continuo $\mathbf{z}_e(i, j)$, se selecciona el embedding más cercano:

$$k^* = \arg\min_k \|\mathbf{z}_e(i, j) - \mathbf{e}_k\|_2^2 \tag{2.5}$$

En vez de actualizar el codebook por gradientes, se utiliza **Exponential Moving Average (EMA)**, que actualiza los embeddings directamente a partir de los vectores del encoder que caen en cada cluster:

$$N_k \leftarrow \gamma N_k + (1-\gamma) n_k \tag{2.6}$$

$$\mathbf{m}_k \leftarrow \gamma \mathbf{m}_k + (1-\gamma) \sum_{i:k_i^*=k} \mathbf{z}_{e,i} \tag{2.7}$$

$$\mathbf{e}_k \leftarrow \frac{\mathbf{m}_k}{N_k} \tag{2.8}$$

donde $\gamma = 0.99$ es el factor de decay y n_k es el número de vectores asignados al embedding k en el batch actual. Las actualizaciones EMA se realizan sin gradientes (`torch.no_grad()`) y usando operaciones in-place para evitar acumulación de memoria.

Straight-through estimator. Como el argmin no es diferenciable, en el backward pass el gradiente se pasa directamente de $\mathbf{z}_q$ a $\mathbf{z}_e$ sin modificación, como si la cuantización fuera una función identidad.

2.3.4. Codebook reset

Un problema común en VQ-VAE es el **colapso del codebook**: el modelo converge a usar solo unos pocos embeddings de los K disponibles. Para evitarlo, se implementa un mecanismo de **codebook reset**: cada 10 steps de entrenamiento, los embeddings con uso muy bajo ($N_k < 1.0$) se reinicializan con vectores aleatorios del batch actual:

$$\mathbf{e}_k \leftarrow \mathbf{z}_{e,\text{random}} \quad \text{si } N_k < \theta_{\text{reset}} \tag{2.9}$$

Este mecanismo permite que la perplexity del codebook alcance valores de ∼90–335 (de 512 posibles), comparado con valores de 3–4 sin codebook reset.

2.3.5. Decoder

El decoder G_θ reconstruye la imagen a partir del mapa cuantizado $\mathbf{z}_q$ mediante capas de convolución transpuesta con stride 2 y activación Tanh final.

2.3.6. Función de pérdida

Con EMA, el término de codebook loss desaparece, quedando:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{commitment}} \tag{2.10}$$

donde $\mathcal{L}_{\text{recon}} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$ y $\mathcal{L}_{\text{commitment}} = \|\mathbf{z}_e - \text{sg}[\mathbf{z}_q]\|_2^2$, con $\beta = 0.25$.

2.3.7. Perplexity del codebook

La perplexity mide cuántos embeddings se están usando efectivamente:

$$\text{perplexity} = \exp\left(-\sum_k \bar{p}_k \log \bar{p}_k\right) \tag{2.11}$$

donde $\bar{p}_k$ es la fracción de vectores asignados al embedding k.

2.4. Clasificación supervisada sobre el espacio latente discreto

2.4.1. Motivación

El VQ-VAE entrenado de forma no supervisada aprende una representación orientada a minimizar el error de reconstrucción, sin ninguna señal que organice los embeddings del codebook por tipo de objeto astronómico. Para estudiar si es posible y beneficioso inducir una organización semántica del codebook, se propone extender el VQ-VAE con una **cabeza de clasificación supervisada** que opera sobre la representación cuantizada $\mathbf{z}_q$.

2.4.2. Cabeza de clasificación sobre $\mathbf{z}_q$

La cabeza de clasificación es un MLP que toma $\mathbf{z}_q \in \mathbb{R}^{D \times H' \times W'}$ y produce logits sobre las $C = 17$ clases de supernovas:

$$\mathbf{z}_q \xrightarrow{\text{GlobalAvgPool}} \mathbb{R}^{256} \xrightarrow{\text{Linear}(256\to128)} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Dropout}} \xrightarrow{\text{Linear}(128\to17)} \mathbb{R}^{17} \tag{2.12}$$

La clasificación se realiza sobre $\mathbf{z}_q$ y no sobre $\mathbf{z}_e$ porque el gradiente fluye hacia el encoder a través del straight-through estimator, presionando al codebook a organizarse semánticamente: embeddings similares quedan asociados a tipos de SN similares. Esto hace los contrafactuales astronómicamente interpretables: al reemplazar tokens anómalos, los tokens de reemplazo corresponden a tipos de SN específicos.

2.4.3. Función de pérdida con clasificación

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{vq}} + \lambda \cdot \mathcal{L}_{\text{class}} \tag{2.13}$$

donde $\mathcal{L}_{\text{class}} = \text{CrossEntropy}(f_\psi(\mathbf{z}_q), y)$ y $\lambda = 1.0$.

2.4.4. Análisis de afinidad del codebook

Para cuantificar la organización semántica del codebook, se define la **afinidad por clase** de cada embedding k:

$$\text{count}(k, c) = \sum_n \sum_{i,j} \mathbf{1}[s_n(i,j) = k] \cdot \mathbf{1}[y_n = c] \tag{2.14}$$

La clase dominante y la pureza de cada embedding son:

$$c^*(k) = \arg\max_c \ \text{count}(k, c), \qquad \text{pureza}(k) = \frac{\text{count}(k,\ c^*(k))}{\sum_c \text{count}(k, c)} \tag{2.15}$$

Esta información se visualiza proyectando los 512 embeddings del codebook con UMAP coloreados por clase dominante, permitiendo comparar directamente la organización antes y después de agregar la señal supervisada.

2.5. Modelo prior sobre el espacio latente discreto

Una vez entrenado el VQ-VAE, cada imagen queda representada como una secuencia de 1024 tokens enteros $\mathbf{s} \in \{1, \ldots, 512\}^{32\times32}$. Se entrena un prior tipo **MaskGIT** (*Masked Generative Image Transformer*) que aprende la distribución conjunta $p(\mathbf{s})$.

2.5.1. Score de anomalía

$$\text{score}(\mathbf{s}) = -\log p(\mathbf{s}) \tag{2.16}$$

Una curva de luz típica produce tokens frecuentes $\rightarrow$ score bajo. Un outlier produce combinaciones de tokens inusuales $\rightarrow$ score alto.

2.5.2. Generación de contrafactuales

Los tokens más improbables se enmascaran y remuestrean condicionalmente:

$$\mathbf{s}' \sim p(\mathbf{s}_{\text{mask}} \mid \mathbf{s}_{\text{visible}}) \tag{2.17}$$

Al decodificar $\mathbf{s}'$ con el decoder del VQ-VAE se obtiene la curva de luz contrafactual, que al compararse con el original revela qué aspectos hacen al objeto anómalo.

Capítulo 3

Dataset

3.1. ELAsTiCC-1

El dataset utilizado corresponde al Extended LSST Astronomical Time Series Classification Challenge (ELAsTiCC-1), desarrollado en el contexto del Observatorio Vera C. Rubin. Contiene curvas de luz simuladas en seis bandas fotométricas: u, g, r, i, z, Y.

3.2. Análisis exploratorio

3.2.1. Distribución de objetos por tipo

El dataset contiene un total de 1.389.147 supernovas distribuidas en 17 tipos.

Tabla 3.1: Distribución de objetos por tipo de supernova en el dataset elasticc_1.

Tipo	**Objetos**	**Obs. total**	**Obs./obj. (media)**	**Obs./obj. (mediana)**
SNIa-SALT2	211.892	23.832.381	112.5	124.0
SNIa-91bg	53.414	6.102.090	114.2	125.0
SNIax	53.727	6.344.784	118.1	126.0
SNII-Templates	103.536	11.298.138	109.1	122.0
SNII-NMF	102.251	12.288.097	120.2	127.0
SNII+HostXT_V19	103.189	11.214.651	108.7	121.0
SNIb-Templates	40.065	4.530.177	113.1	125.0
SNIb+HostXT_V19	102.883	11.287.051	109.7	122.0
SNIc-Templates	20.617	2.383.542	115.6	125.0
SNIc+HostXT_V19	103.885	11.425.289	110.0	123.0
SNIcBL+HostXT_V19	42.878	4.774.667	111.4	123.0
SNIIb+HostXT_V19	103.683	11.294.039	108.9	122.0
SNIIn-MOSFIT	31.162	3.905.954	125.3	130.0
SNIIn+HostXT_V19	105.281	12.285.982	116.7	125.0
SLSN-I+host	100.977	15.814.118	156.6	163.0
SLSN-I_no_host	4.261	761.439	178.7	193.0
PISN	105.446	14.563.011	138.1	139.0
Total	**1.389.147**	**163.905.410**	120.4	126.0

Se observa un desbalance importante: el tipo más abundante (SNIa-SALT2, 211.892 ob-

jetos) tiene 50 veces más objetos que el tipo más escaso (SLSN-I_no_host, 4.261 objetos). Este desbalance es relevante para la detección de anomalías, ya que tipos poco frecuentes podrían ser detectados como outliers simplemente por su rareza.

3.2.2. Características temporales

Todos los tipos comparten el mismo rango temporal de 1.104,3 días (3 años). Destacan dos grupos:

- **Supernovas estándar** (SNIa, SNII, SNIb, SNIc): entre 108 y 127 observaciones por objeto en promedio.
- **Supernovas superluminosas** (SLSN-I) y **PISN**: entre 138 y 178 observaciones por objeto, reflejando su mayor duración y brillo.

3.2.3. Objetos potencialmente anómalos

Desde el punto de vista de la detección de anomalías sin supervisión, destacan como candidatos naturales a outliers:

- **SLSN-I_no_host** (4.261 objetos): el tipo más escaso, con curvas de luz significativamente más largas (mediana 193 observaciones).
- **PISN** (105.446 objetos): supernovas de inestabilidad de par, con morfología particular.
- **SLSN-I+host** (100.977 objetos): supernovas superluminosas con curvas muy extendidas (mediana 163 observaciones).

3.3. Preprocesamiento: imágenes 2grid

A partir del dataset se generaron 895.354 imágenes 2grid de 256×256 píxeles. La generación se realizó en paralelo en el cluster Leftraru HPC, lanzando un job SLURM independiente por cada tipo de supernova. Las imágenes se almacenan en `data/images/elasticc_1/2grid/{SNID}.png`.

Para el entrenamiento del VQ-VAE se utilizaron 899.440 imágenes divididas en:

- **Training:** 718.673 imágenes (80 %, fold 0)
- **Validación:** 179.669 imágenes (20 %, fold 0)

La partición utiliza 5-fold cross-validation estratificada por tipo de supernova (`StratifiedKFold`, `random_state=42`).

Capítulo 4

Metodología

La metodología se organiza en tres etapas secuenciales: preprocesamiento de los datos como imágenes, entrenamiento del VQ-VAE, y detección de anomalías con generación de contrafactuales.

4.1. Exploración metodológica previa

Antes de adoptar el enfoque basado en imágenes, se realizó una exploración inicial con series de tiempo directamente como input del VQ-VAE, siguiendo el esquema de TimeVQVAE [**?**]. La transformada de Fourier no resultó adecuada para supernovas (muestreo irregular, características morfológicas no frecuenciales), por lo que se trabajó directamente con la serie de tiempo.

Se realizaron dos corridas variando el tamaño del codebook:

Tabla 4.1: Resultados de la exploración inicial con series de tiempo.

Métrica	**codebook_dim=8**	**codebook_dim=64**
Perplexity (train)	1.00	5.23
Recon loss (train)	0.25012	0.02683

La perplexity de 1.00 indica colapso total del codebook. A pesar de que la calidad de reconstrucción mejoró con `codebook_dim=64`, las reconstrucciones presentaban artefactos visibles en los pasos iniciales y no capturaban fielmente la morfología de las curvas de luz, como se muestra en las Figuras **??** y **??**.

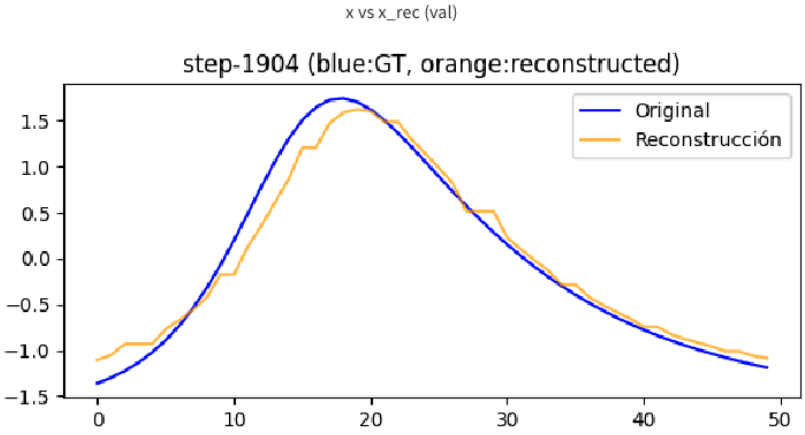


Figura 4.1: Reconstrucción del VQ-VAE sobre series de tiempo en el step 1904. Azul: curva original. Naranja: reconstrucción.

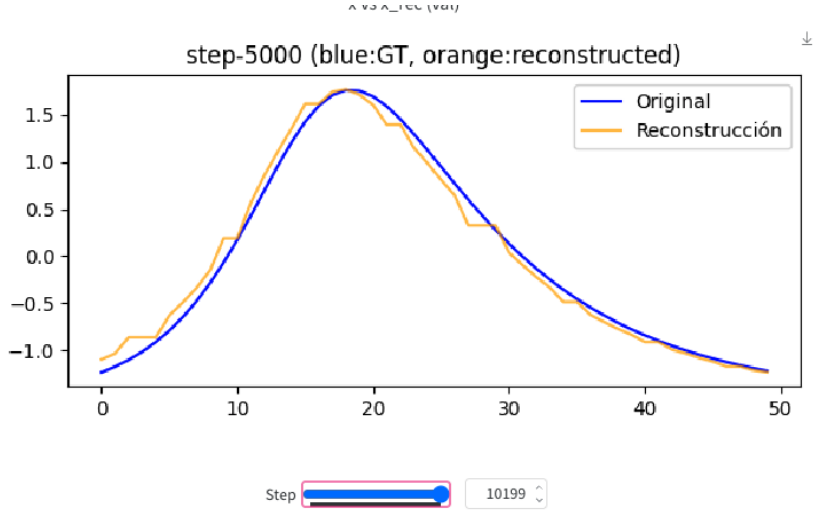


Figura 4.2: Reconstrucción del VQ-VAE sobre series de tiempo en el step 5000. La reconstrucción mejora pero presenta desplazamientos temporales.

Estos resultados, junto con las limitaciones del muestreo irregular, motivaron el cambio a representación en imágenes.

4.2. Preprocesamiento: conversión de curvas de luz a imágenes 2grid

Las curvas de luz se convierten en imágenes RGB de 256×256 píxeles con 6 bandas apiladas verticalmente.

4.2.1. Normalización por objeto

Para cada supernova, el tiempo MJD y el flujo fotométrico se normalizan individualmente al rango $[0, 1]$:

$$\hat{t} = \frac{t - t_{\min}}{t_{\max} - t_{\min}}, \qquad \hat{f} = \frac{f - Q_{0.01}}{Q_{0.99} - Q_{0.01}} \tag{4.1}$$

Con esta normalización, las imágenes tienen $\sim$10 % de píxeles con contenido, frente al 4 % sin normalización.

4.2.2. Normalización para el modelo

Antes de ser procesadas por el VQ-VAE, las imágenes se normalizan a $[-1, 1]$:

$$\hat{x} = \frac{x - 0.5}{0.5} \tag{4.2}$$

Consistente con la activación Tanh del decoder.

4.3. Entrenamiento del VQ-VAE

4.3.1. Arquitectura

- **Encoder:** tres capas Conv2d con stride 2 (256×256 → 32×32) + dos ResBlocks. Salida: $\mathbf{z}_e \in \mathbb{R}^{256\times32\times32}$.
- **Codebook EMA:** 512 embeddings de dimensión 256, decay=0.99. Codebook reset cada 10 steps para embeddings con $N_k < 1.0$.
- **Decoder:** espejo del encoder con ConvTranspose2d + activación Tanh.

El modelo tiene $\sim$7.7 millones de parámetros entrenables.

4.3.2. Configuración del entrenamiento

Tabla 4.2: Hiperparámetros del entrenamiento del VQ-VAE.

Hiperparámetro	**Valor**
Batch size	32
Learning rate	2×10^{-4}
Optimizador	Adam
Commitment cost (β)	0.25
Tamaño del codebook (K)	512
Dimensión del embedding (D)	256
Resolución latente	32×32
EMA decay	0.99
Codebook reset threshold	1.0
Precisión	float16 mixta
Gradient clip	1.0

4.4. Clasificación supervisada sobre el codebook

4.4.1. Motivación

La visualización mediante UMAP de los tokens generados mostró que las clases de supernovas no se separan claramente en el espacio discreto aprendido con reconstrucción sola. Para investigar si es posible inducir organización semántica del codebook, se incorporó una cabeza de clasificación supervisada sobre $\mathbf{z}_q$.

4.4.2. Función de pérdida extendida

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{vq}} + \lambda \cdot \mathcal{L}_{\text{class}} \tag{4.3}$$

Tabla 4.3: Hiperparámetros de la cabeza de clasificación.

Hiperparámetro	**Valor**
Número de clases	17
Peso de clasificación (λ)	1.0
Dropout clasificador	0.1

4.4.3. Resultados preliminares (epoch 0 en curso)

Al inicio del entrenamiento con clasificación, el modelo muestra:

- **cls_loss**: parte en ∼2.83 (entropía máxima para 17 clases) y desciende a ∼2.29 en los primeros 2.000 steps.
- **acc**: sube de 5.9 % (random) a ∼31 % en los primeros 2.000 steps.
- **perplexity**: ∼335, significativamente más alta que el modelo base (∼162), indicando que la señal de clasificación no colapsa el codebook sino que mantiene alta diversidad.
- **recon_loss**: ∼0.012, comparable con el modelo base al final del epoch 0.

4.5. Detección de anomalías

Una vez entrenado el VQ-VAE, cada supernova queda representada como $\mathbf{s} \in \{1, \ldots, 512\}^{32\times32}$. Se entrena un prior MaskGIT sobre las secuencias de tokens del conjunto de entrenamiento y se calcula el score de anomalía:

$$\text{score}(\mathbf{s}) = -\log p(\mathbf{s}) \tag{4.4}$$

4.6. Generación de contrafactuales

Los tokens más improbables se enmascaran y remuestrean condicionalmente:

$$\mathbf{s}' \sim p(\mathbf{s}_{\text{mask}} \mid \mathbf{s}_{\text{visible}}) \tag{4.5}$$

La calidad se evalúa según plausibilidad, parsimonia y consistencia temporal entre bandas.

Capítulo 5

Resultados

5.1. VQ-VAE base

5.1.1. Calidad de reconstrucción

El VQ-VAE base fue entrenado durante 16 epochs sobre 718.673 imágenes 2grid del dataset ELAsTiCC-1. Al término del entrenamiento, la pérdida total sobre el conjunto de entrenamiento alcanzó 0.0003, con una pérdida de reconstrucción de 0.000216. La perplexity efectiva del codebook se situó en ∼162 durante el entrenamiento y ∼196 sobre el conjunto de validación, lo que indica que entre 162 y 196 de los 512 embeddings disponibles participan activamente en la representación del dataset (Tabla **??**).

Tabla 5.1: Resultados del VQ-VAE base al finalizar el entrenamiento.

Métrica	**Valor**
train/loss (mejor checkpoint)	0.0003
train/recon_loss	0.000216
train/perplexity	∼162
val/perplexity	∼196

5.1.2. Análisis del espacio latente discreto

Para caracterizar la organización interna del espacio latente aprendido, se proyectaron los tokens generados sobre una muestra del conjunto de validación mediante UMAP, asignando a cada punto el color correspondiente al tipo de supernova de la imagen de origen. La Figura **??** presenta el resultado.

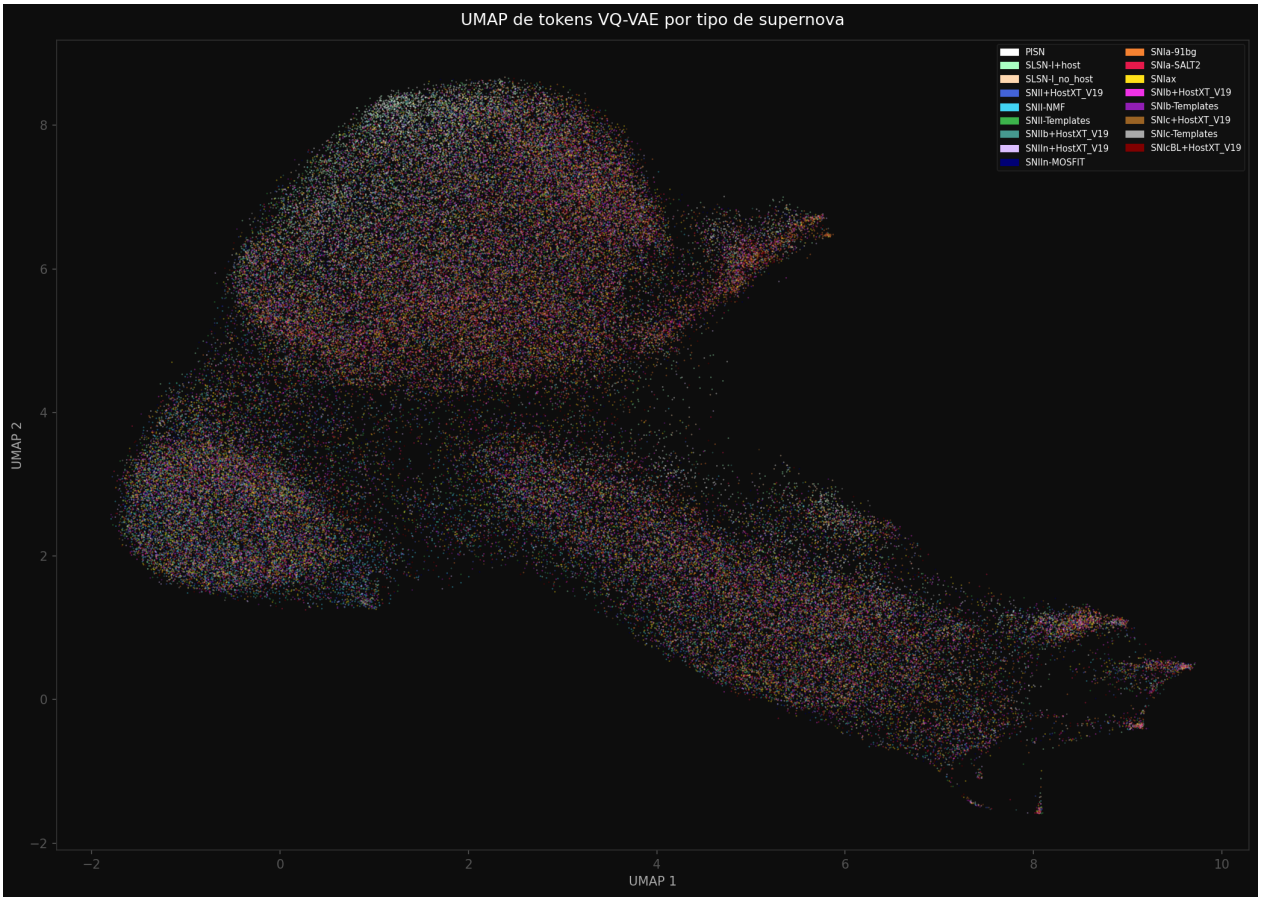


Figura 5.1: UMAP de tokens del VQ-VAE base coloreados por tipo de supernova. Cada punto corresponde a un token generado por una imagen 2grid del conjunto de validación.

La proyección revela una estructura geométrica con masa central compacta y varios brazos periféricos. Los 17 tipos de supernovas se distribuyen de forma mezclada a lo largo de toda la estructura, sin formar clusters diferenciados por clase. Para cuantificar esta distribución, se identificaron cinco regiones geográficas en el UMAP y se calculó la composición de tipos dentro de cada una (Figura **??** y Figura **??**).

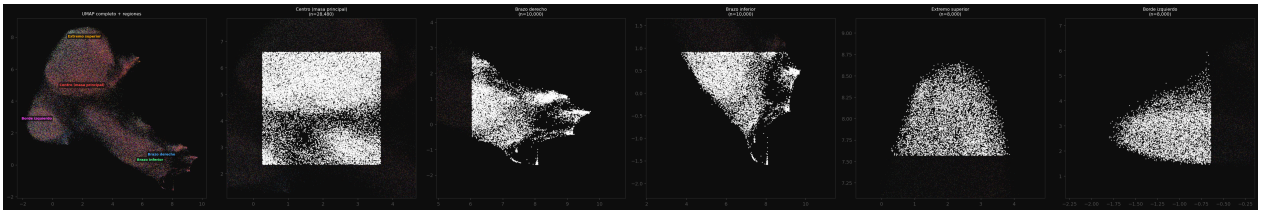


Figura 5.2: Delimitación de las cinco regiones identificadas en el espacio UMAP: masa central, brazo derecho, brazo inferior, extremo superior y borde izquierdo.

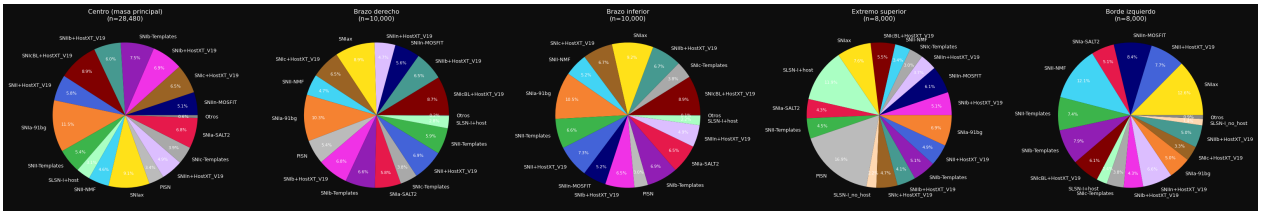


Figura 5.3: Composición por tipo de supernova en cada región del UMAP. La distribución de clases es similar entre zonas, sin dominancia de ningún tipo particular.

Ninguna región está dominada por un tipo particular y la mezcla de clases es similar entre zonas, con variaciones menores que se explican por el desbalance del dataset. La Figura **??** muestra imágenes 2grid representativas de cada región: las curvas de luz varían morfológicamente —el brazo derecho agrupa curvas con peaks más estrechos, la masa central concentra morfologías más extendidas— pero esa variación no coincide con los límites entre tipos astronómicos.

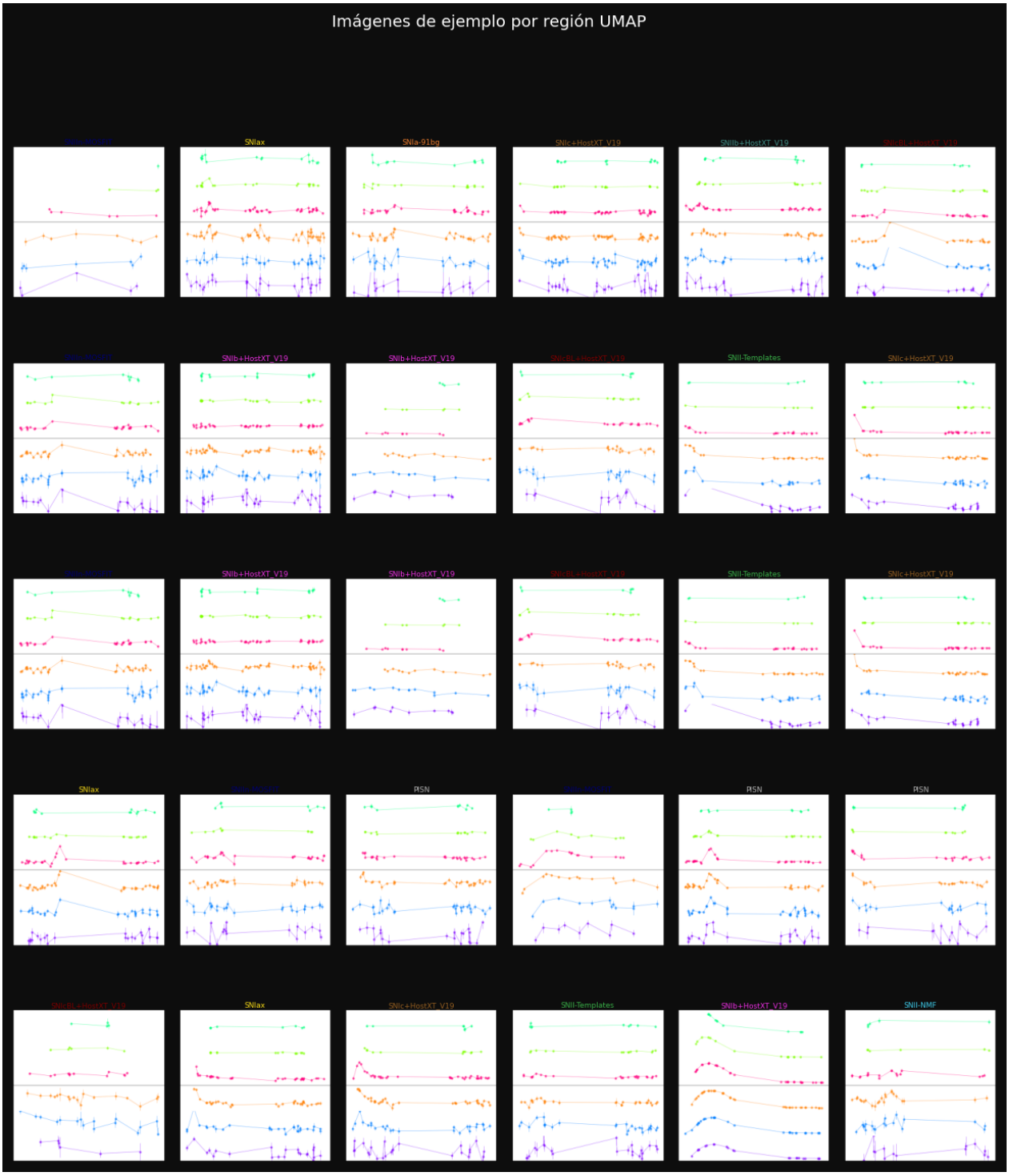


Figura 5.4: Imágenes 2grid de ejemplo por región del UMAP. Los títulos en color indican el tipo de supernova de cada imagen.

El VQ-VAE base organiza el espacio latente por morfología visual de la curva de luz, no por tipo de objeto. La clasificación por tipo requiere una señal supervisada explícita, lo que justifica la extensión descrita en la sección siguiente.

5.2. VQ-VAE con clasificación supervisada

5.2.1. Entrenamiento

El VQ-VAE extendido con un módulo de clasificación supervisada (*classification head*) sobre $\mathbf{z}_q$ se entrenó sobre el mismo conjunto de 718.673 imágenes, incorporando la pérdida de clasificación con peso $\lambda = 0.1$. El modelo converge en 7 epochs, alcanzando una exactitud de validación de 55.6 % y una pérdida total de 0.1188 en el mejor checkpoint. La exactitud sobre el conjunto completo de validación (109.737 imágenes) es 38.8 %, reflejo del desbalance entre tipos: la matriz de confusión muestra un recall de 0.88 para PISN y 0.81 para SLSN-I+host, mientras SNIIb y SNIc-Templates no superan 0.02 (Figura **??**).

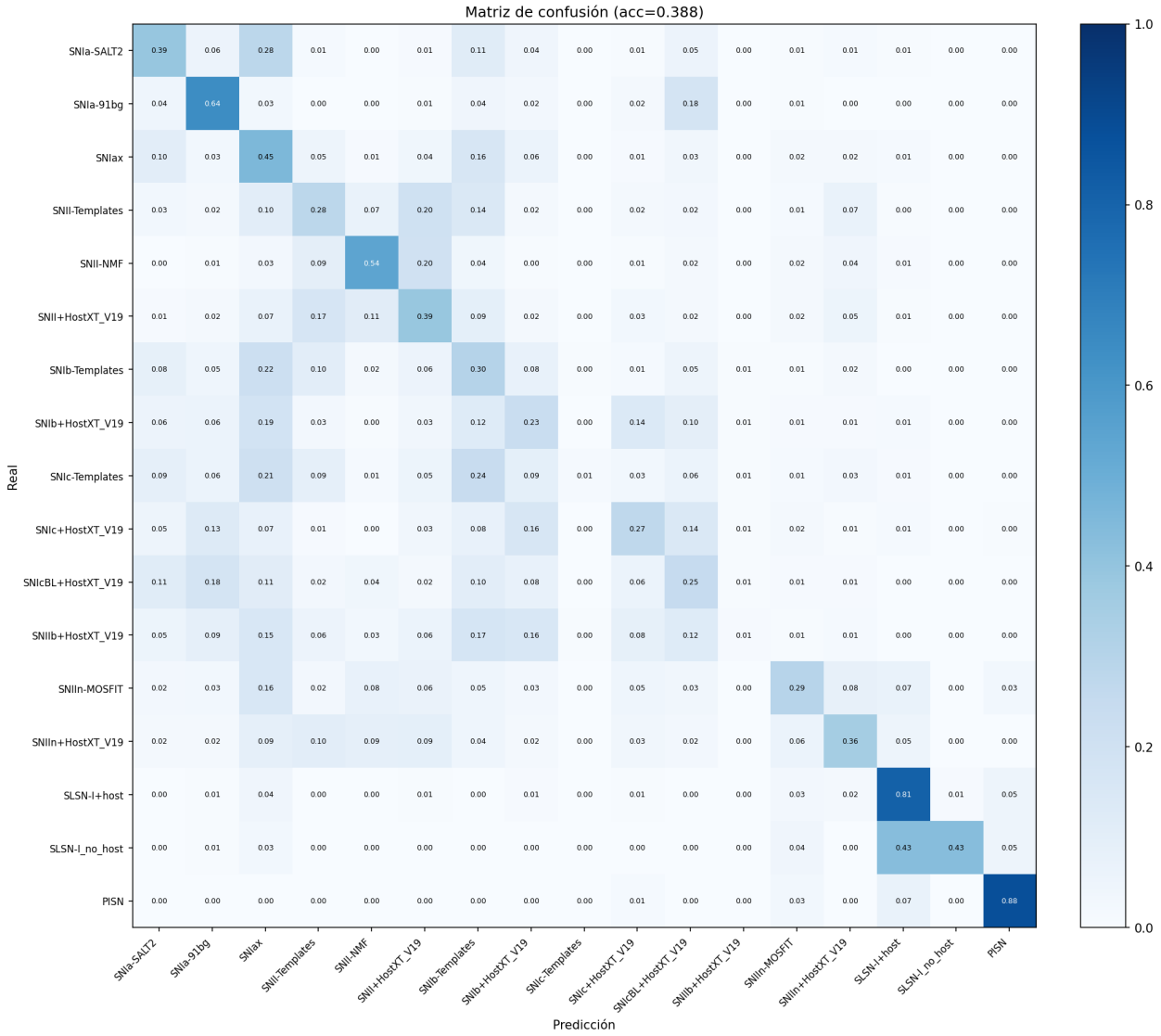


Figura 5.5: Matriz de confusión del módulo de clasificación supervisada sobre el conjunto de validación (109.737 imágenes, 17 clases). Las filas corresponden a la clase verdadera y las columnas a la clase predicha. PISN y SLSN-I+host concentran la mayor exactitud por clase; los tipos morfológicamente similares entre sí (SNIIb, SNIc-Templates) presentan la mayor tasa de confusión.

5.2.2. Análisis del codebook con señal supervisada

Con 512 embeddings activos sobre 512 totales, el codebook se utiliza por completo, a diferencia del modelo base donde la perplexity efectiva era de ∼162–196. La pureza media del codebook es 0.154: con 17 clases, la pureza aleatoria esperada sería $1/17 \approx 0.059$, de modo que los embeddings aprendidos son ∼2.6 veces más puros que el azar, sin llegar a una especialización fuerte.

La Figura **??** presenta la proyección UMAP de los 512 embeddings del codebook, coloreada por clase dominante (izquierda) y por pureza (derecha).

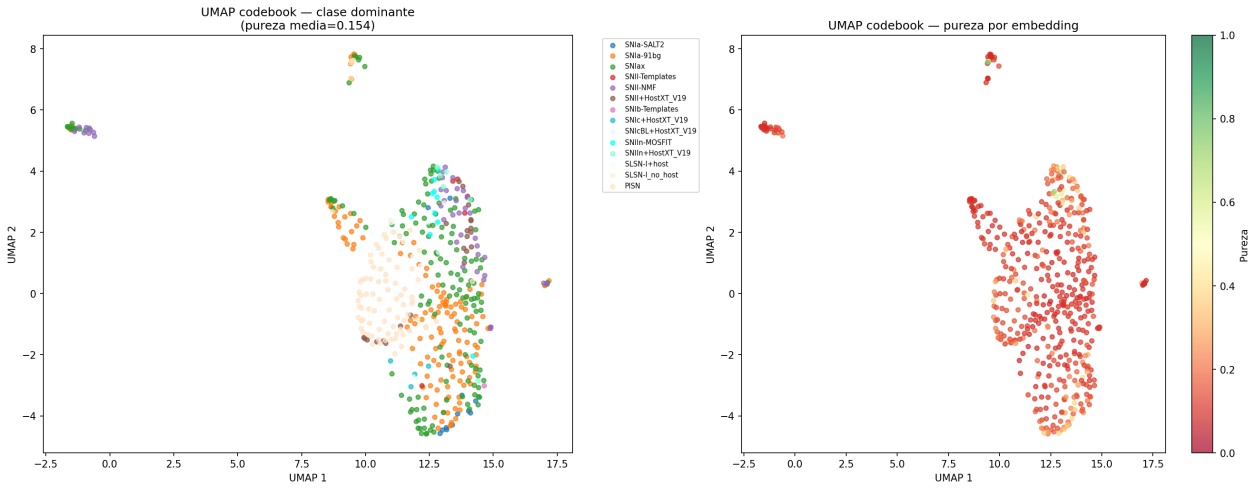


Figura 5.6: UMAP de los 512 embeddings del codebook tras el entrenamiento con clasificación supervisada ($\lambda = 0.1$). Izquierda: color por clase dominante de cada embedding. Derecha: color por pureza (pureza(k) = count$(k, c^*)/\sum_c$ count(k, c)). Pureza media: 0.154.

La estructura del UMAP revela dos comportamientos distintos. El cluster principal agrupa la mayoría de los embeddings con pureza baja (0.1–0.3), donde los tipos de SN estándar (SNIa, SNII, SNIb, SNIc) coexisten sin separación clara. Fuera de este cluster principal aparecen dos grupos aislados de alta pureza (≥ 0.8): uno dominado por SLSN-I y otro por PISN. Estos dos tipos son los morfológicamente más distintos del resto del dataset —curvas de mayor duración y brillo— y son precisamente los que el módulo de clasificación identifica con mayor recall (0.81 y 0.88 respectivamente).

5.2.3. Comparación con el modelo base

El VQ-VAE base organiza el espacio latente por morfología visual de la curva de luz, sin correspondencia con los tipos astronómicos (Sección **??**). La señal supervisada desplaza parcialmente esa organización: los tipos más atípicos quedan aislados en embeddings dedicados de alta pureza, mientras los tipos estándar —cuyas curvas de luz son morfológicamente similares entre sí— permanecen mezclados.

Este resultado es consistente con la exactitud diferencial del módulo de clasificación: $\lambda = 0.1$ alcanza para separar PISN y SLSN-I, pero no para discriminar tipos con morfologías parecidas como SNIIb y SNIc. La organización semántica del codebook refleja directamente la dificultad intrínseca del problema de clasificación sobre representaciones visuales de curvas de luz.

5.3. Prior MaskGIT

5.4. Detección de anomalías

5.5. Generación de contrafactuales

Capítulo 6

Discusión y Conclusiones

6.1. Discusión

6.1.1. Representación 2grid

La conversión de curvas de luz multibanda a imágenes 2grid resultó efectiva para el entrenamiento del VQ-VAE. La normalización por objeto fue un aspecto crítico: sin ella, las curvas quedan fuera del área visible de la imagen (4 % de píxeles con contenido), mientras que con normalización se alcanza 10 %, lo que garantiza que el modelo aprenda patrones morfológicos y no valores absolutos de flujo.

6.1.2. Organización semántica del codebook

6.1.3. Calidad de los contrafactuales

6.2. Limitaciones

- El pipeline fue evaluado sobre datos simulados (ELAsTiCC-1). La generalización a datos reales del Observatorio Vera C. Rubin requiere validación adicional.
- El desbalance entre tipos de supernovas (hasta 50x entre el tipo más y menos frecuente) puede sesgar la distribución aprendida por el prior MaskGIT.

6.3. Conclusiones

6.4. Trabajo futuro

- Evaluar distintos valores de λ para el peso de la clasificación y su efecto en el downstream de anomaly detection.
- Explorar tamaños de codebook alternativos ($K = 256$, $K = 1024$) y su efecto en la calidad de los contrafactuales.
- Aplicar el pipeline a datos reales del Observatorio Vera C. Rubin cuando estén disponibles.